





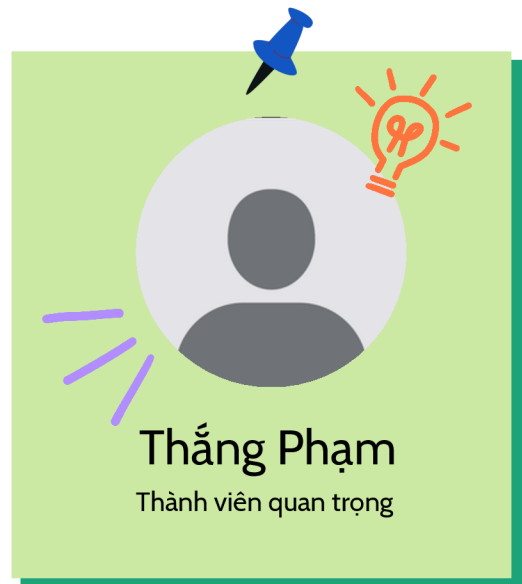
Công Trí

Thành viên nền móng

Giới thiệu

Nhiệm vụ :

- ✓ Tìm hiểu thuật toán A*
- ✓ Triển khai thuật toán A*
- ✓ Phụ trách latex



Giới thiệu

Nhiệm vụ :

- ✓ Cài đặt thuật toán A*
- ✓ Phụ trách slide



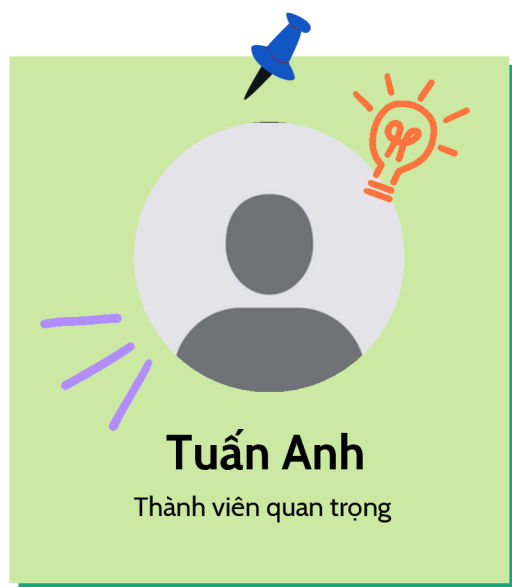
Đặng Quang

Thành viên quan trọng

Giới thiệu

Nhiệm vụ :

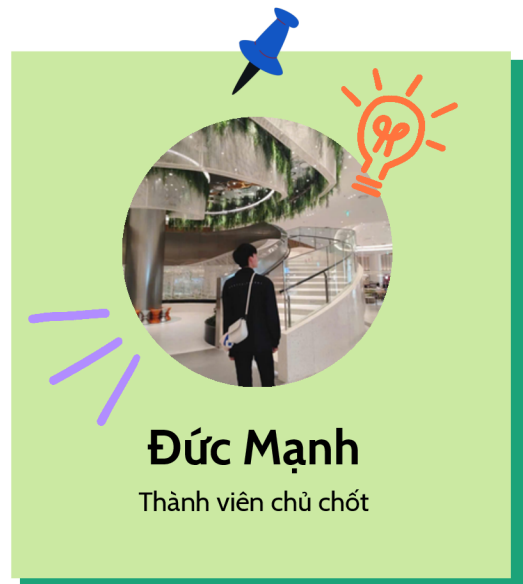
- ✓ Mã nguồn
- ✓ Phụ trách slide



Giới thiệu

Nhiệm vụ :

- ✓ Tổng hợp nội dung
- ✓ Kiểm tra và chỉnh sửa
- ✓ Phụ trách slide



Mục lục

GIỚI THIỆU BÀI
TOÁN 8-PUZZLE

Nguyên lý
thuật toán A*

Cài đặt thuật toán
A* cho bài toán
8-puzzle

Mã Nguồn

Lưu ý : sản phẩm là tâm huyết của nhóm

GIỚI THIỆU BÀI TOÁN 8-PUZZLE

Trò chơi 8 quân cờ thường chơi trên một bảng vuông 3x3 với 8 ô và một ô trống. Mục tiêu của trò chơi là sắp xếp các ô số từ 1 đến 8 theo một thứ tự cụ thể hoặc theo một trạng thái mà người chơi muốn di chuyển mỗi ô số hoặc ô trống sang một ô trống khác, nhưng chỉ có thể di chuyển 1 bước một lần

Ta có thể áp dụng các thuật toán tìm kiếm như A*, Tìm kiếm theo chiều sâu (DFS), tìm kiếm theo chiều rộng (BFS) để đánh giá và lựa chọn các nước di chuyển.



1	2	3
4	5	6
7	8	

Phiếu bài tập

Phần I: Trách nhiệm

Câu 1. Chỉ tiêu nào sau đây không thuộc chỉ tiêu ngoại hình khi chọn giống vật nuôi?

- A. Tầm vóc**
- B. Màu sắc lông, da**
- C. Năng suất sữa**
- D. Thể chất**



Câu 2. Trong chọn giống vật nuôi, chỉ tiêu sinh trưởng thường được đánh giá dựa vào:

- A. Khả năng sinh sản của con cái**
- B. Tốc độ tăng khối lượng cơ thể**
- C. Chất lượng sản phẩm thịt, trứng**
- D. Tỷ lệ mỡ cơ thể**

Câu 3. Chỉ tiêu nào sau đây thuộc chỉ tiêu sinh sản?

- A. Chiều cao, vòng ngực**
- B. Năng suất lông cừu**
- C. Số con sơ sinh sống/cái/lứa**
- D. Tốc độ tăng trọng**



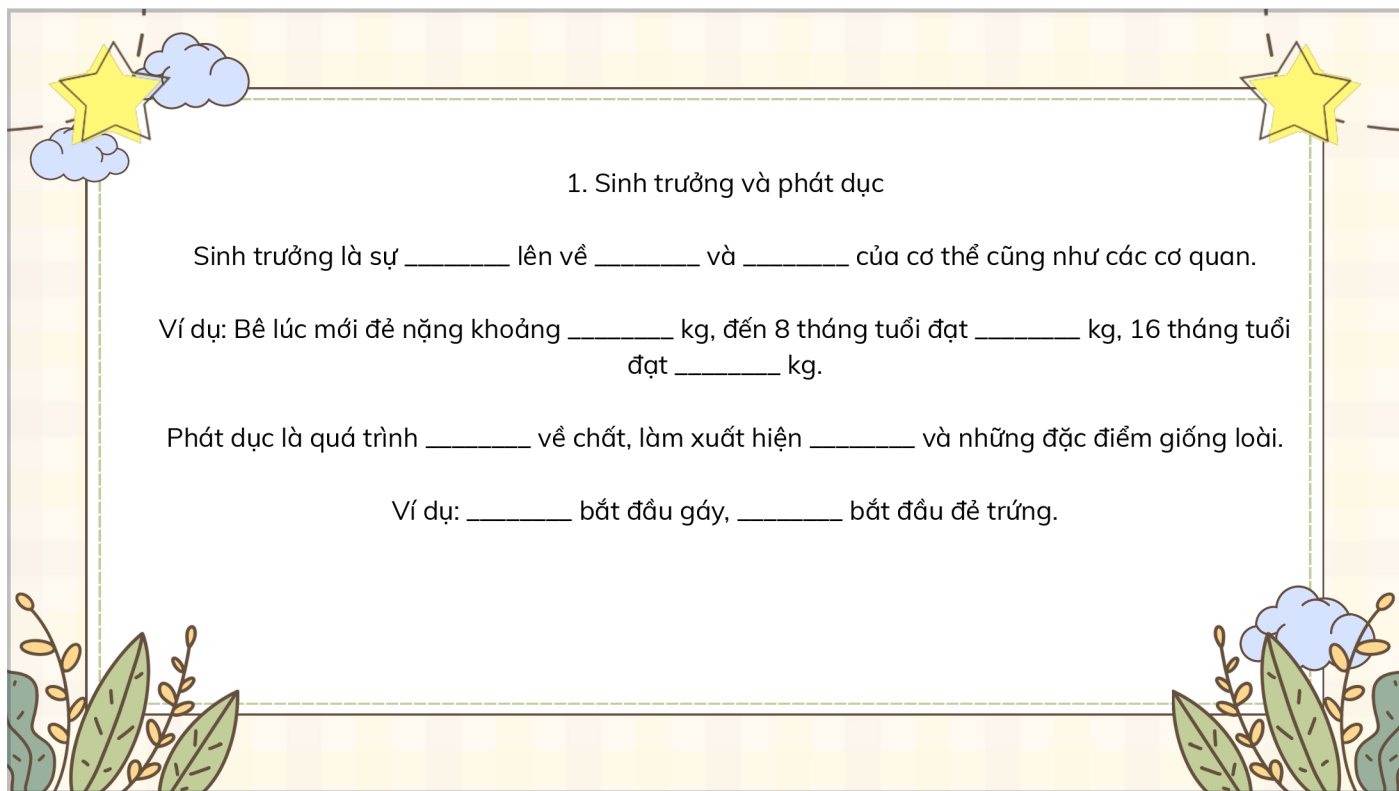


Câu 4. Năng suất trứng của gà mái thường được tính theo:

- A. Trọng lượng trứng**
- B. Số quả trứng/mái/năm**
- C. Màu sắc vỏ trứng**
- D. Độ dày vỏ trứng**

Câu 5. Ở vật nuôi lấy thịt, chỉ tiêu sản xuất thường được đánh giá thông qua:

- A. Tốc độ sinh trưởng, tỉ lệ thịt xẻ**
 - B. Số con sinh ra/lứa**
 - C. Màu lông và ngoại hình**
 - D. Lượng sữa tiết ra trong ngày**
- 
- 





Phần II: Câu trắc nhiệm đúng sai

Câu 1:

- 1. Khả năng sản xuất của vật nuôi chỉ phụ thuộc vào giống.**
 - 2. Tốc độ sinh trưởng là một chỉ tiêu đánh giá khả năng sản xuất của vật nuôi.**
 - 3. Chọn lọc giống không ảnh hưởng đến khả năng sản xuất của vật nuôi.**
 - 4. Dinh dưỡng hợp lý không cần thiết cho vật nuôi.**
- 
- 

NGUYÊN LÝ HOẠT ĐỘNG



Thuật toán A* hoạt động thông qua một quy trình lặp lại, liên tục mở rộng các nút trong đồ thị cho đến khi tìm ra được đường đi tối ưu. Quy trình này có thể được chia thành các bước chính như sau:

Khởi tạo :

Thuật toán A* bắt đầu bằng việc đưa điểm bắt đầu vào danh sách mở (open list). Mỗi điểm trong danh sách này sẽ chứa các thông tin về chi phí $g(n)$, hàm heuristic $h(n)$ và tổng chi phí $f(n)$.

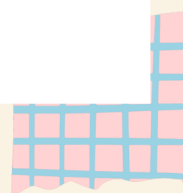
NGUYÊN LÝ HOẠT ĐỘNG



Lặp lại tìm kiếm :

Mỗi vòng lặp, thuật toán A^* sẽ chọn điểm có giá trị $f(n)$ nhỏ nhất trong danh sách mở. Điểm này được gọi là nút hiện tại và thuật toán sẽ kiểm tra tất cả các điểm kế tiếp có thể di chuyển đến từ nút này. Sau khi kiểm tra, các điểm này sẽ được đưa vào:

- Danh sách đóng (closed list): Nếu điểm đó đã được kiểm tra trước, không cần kiểm tra lại.
- Danh sách mở (open list): Nếu điểm đó chưa được kiểm tra, sẽ được thêm vào danh sách mở để tiếp tục xử lý sau này.



NGUYÊN LÝ HOẠT ĐỘNG



Tính toán $f(n)$:

Khi thuật toán lựa chọn một điểm trong danh sách mở, nó sẽ tính toán giá trị $f(n)$ cho các điểm kế tiếp (nút con) dựa trên công thức $f(n) = g(n) + h(n)$. Sau đó, giá trị này sẽ được so sánh với giá trị $f(n)$ của điểm đã được kiểm tra trước đó.

Nếu đường đi mới đến một điểm có chi phí tổng $f(n)$ thấp hơn so với đường đi cũ, thuật toán sẽ cập nhật lại thông tin và lựa chọn đường đi mới này. Điều này giúp thuật toán tìm ra đường đi tối ưu và tối thiểu hóa chi phí.



Kết thúc:

Quá trình tìm kiếm tiếp tục cho đến khi một trong hai điều kiện sau xảy ra:

- Thuật toán tìm thấy điểm đích và đường đi tối ưu được xác định.
- Danh sách mở trống, tức là không còn điểm nào khả thi để kiểm tra và không có đường đi nào dẫn đến đích.

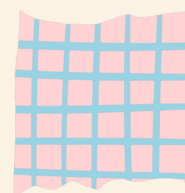


NGUYÊN LÝ HOẠT ĐỘNG



KẾT LUẬN :

Điều khiến thuật toán A^* trở nên hiệu quả và mạnh mẽ chính là việc sử dụng hàm heuristic $h(n)$. Nếu hàm này được thiết kế hợp lý, thuật toán A^* sẽ có thể tìm ra đường đi tối ưu trong thời gian rất ngắn. Đặc biệt, nếu hàm heuristic là “thực tế (admissible)”, tức là không bao giờ ước lượng chi phí quá cao so với giá trị thực tế, thuật toán A^* sẽ luôn tìm ra được giải pháp tối ưu.



CẢI ĐẶT THUẬT TOÁN



Các bước tiến hành cài đặt A* cho bài toán 8-puzzle

Bước 1: Xác định đầu vào, đầu ra và điều kiện dừng

- Đầu vào: trạng thái xuất phát và trạng thái đích.
- Đầu ra: chuỗi các bước di chuyển để đưa trạng thái ban đầu về trạng thái đích.
- Điều kiện dừng: khi thuật toán tìm thấy trạng thái đích



Bước 2: Khởi tạo cấu trúc dữ liệu

- Open list: chứa các trạng thái đã sinh ra nhưng chưa mở rộng
- Closed list: chứa các trạng thái đã được mở rộng.
- Mỗi trạng thái lưu trữ thông tin $g(n)$, $h(n)$, $f(n)$ cùng con trỏ về trạng thái cha để truy vết lời giải

Bước 3: Chọn hàm heuristic

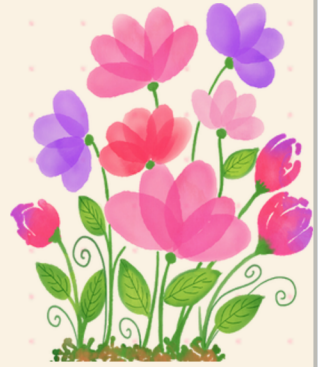
- Số ô sai vị trí (Misplaced tiles): đơn giản, nhanh, nhưng độ chính xác thấp.
- Khoảng cách Manhattan: tính tổng khoảng cách theo trục ngang và dọc của mỗi ô đến vị trí đích. Mặc dù tính toán phức tạp hơn nhưng cho kết quả chính xác hơn, giúp A* tìm kiếm với ít bước hơn

Bước 4: Tính toán hàm đánh giá

$$f(n) = g(n) + h(n)$$

Trong đó:

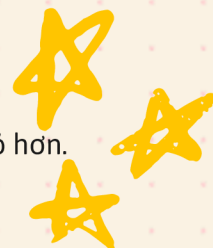
- $g(n)$: chi phí thực tế từ trạng thái ban đầu đến trạng thái hiện tại.
- $h(n)$: chi phí ước lượng từ trạng thái hiện tại đến trạng thái đích.



Bước 5: Thực hiện thuật toán A*



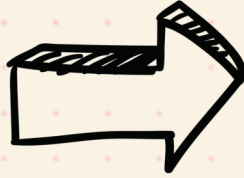
- Step 1: Khởi tạo open list với trạng thái ban đầu, closed list rỗng
- Step 2: Lặp lại cho đến khi open list rỗng:
 - + Nếu trạng thái hiện tại trùng với trạng thái đích → dừng thuật toán, truy vết đường đi
 - + Lấy trạng thái có $f(n)$ nhỏ nhất từ open list → đây là trạng thái hiện tại.
 - + Nếu chưa phải trạng thái đích: đưa trạng thái hiện tại vào closed list, sinh ra các trạng thái con (di chuyển ô trống lên, xuống, trái, phải nếu hợp lệ).
 - + Với mỗi trạng thái con:
 - Tính $g(n), h(n)$
 - Nếu con đã nằm trong closed list → bỏ qua.
 - Nếu con đã nằm trong open list → so sánh và giữ lại trạng thái có $f(n)$ nhỏ hơn.
 - Nếu con chưa có trong open list và closed list → thêm vào open list.
 - Nếu con chưa có trong open list và closed list → thêm vào open list.
- Step 3: Nếu open list rỗng mà chưa tìm thấy trạng thái đích → kết luận không tồn tại đường đi.



Minh họa tính toán heuristic

Giả sử có trạng thái hiện tại

2	8	3
1	6	4
7		5



Trạng thái đích

1	2	3
4	5	6
7	8	



Heuristic 1 – số ô sai vị trí

- Các ô sai vị trí gồm 2, 8, 1, 6, 4, 5 → tổng cộng 6 ô.
- Vậy $h(n)=6$
- Đối với khoảng cách Mahattan, đây là một hàm heuristic chính xác hơn, tính toán tổng khoảng cách theo trục ngang và trục dọc của mỗi ô từ vị trí hiện tại đến vị trí đích của nó. Cách tính như sau: tính khoảng cách Mahatan cho từng ô bằng cách lấy tổng số bước di chuyển theo hàng và cột để đưa ô số từ vị trí hiện tại đến vị trí đích



Heuristic 2 – khoảng cách Manhattan

- Ô 2: từ (0,0) → (0,1) = 1 bước
- Ô 8: từ (0,1) → (2,1) = 2 bước
- Ô 3: đúng vị trí = 0 bước
- Ô 1: từ (1,0) → (0,0) = 1 bước
- Ô 6: từ (1,1) → (1,2) = 1 bước
- Ô 4: từ (1,2) → (1,1) = 1 bước
- Ô 7: đúng vị trí = 0 bước
- Ô 5: từ (2,2) → (1,2) = 1 bước

→ Tổng khoảng cách Manhattan: $1 + 2 + 0 + 1 + 1 + 1 + 0 + 1 = 7$

Kết luận

Heuristic “số ô sai vị trí” đơn giản nhưng kém chính xác. Heuristic “khoảng cách Manhattan” tuy tốn công tính toán hơn nhưng ước lượng sát hơn, giúp A* mở rộng ít trạng thái hơn và đạt được lời giải tối ưu

MÃ NGUỒN

I. Khai báo ban đầu

```
import heapq

TrangThaiDich = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 0]]
```

- Sử dụng heapq để cài đặt hàng đợi ưu tiên (priority queue), cần thiết cho thuật toán A* (chọn trạng thái có $f = g + h$ nhỏ nhất để mở rộng).
- TrangThaiDich: ma trận 3x3 biểu diễn trạng thái đích
 - +) Số 0 đại diện cho ô trống.
 - +) Các số từ 1 đến 8 là các ô gạch.



II. Hàm tìm vị trí giá trị trong ma trận

```
def TimViTri(TrangThai, GiaTri):  
    for i in range(3):  
        for j in range(3):  
            if TrangThai[i][j] == GiaTri:  
                return i, j  
    return None
```



Trả về (i, j) vị trí của số GiaTri trong ma trận.

III. Hàm heuristic: Khoảng cách Manhattan



```
def KhoangCachManhattan(TrangThai):  
    tong = 0  
    for i in range(3):  
        for j in range(3):  
            GiaTri = TrangThai[i][j]  
            if GiaTri != 0: # bỏ qua ô trống  
                iDich, jDich = TimViTri(TrangThaiDich, GiaTri)  
                tong += abs(i - iDich) + abs(j - jDich)  
    return tong
```



Tính tổng khoảng cách Manhattan của các ô đến vị trí đúng trong TrangThaiDich.

IV. Chuyển ma trận sang tuple

```
def DoiSangTuple(TrangThai):  
    return tuple(tuple(hang) for hang in TrangThai)
```

Dùng set để lưu trạng thái đã duyệt (sau khi chuyển sang tuple), nhằm tránh lặp lại và giảm thời gian tìm kiếm

IV. Sinh trạng thái hàng xóm

```
def HangXom(TrangThai):  
    KetQua = []  
    x, y = TimViTri(TrangThai, 0)  
    BuocDi = [(-1,0,"Lên"), (1,0,"Xuống"), (0,-1,"Trái"), (0,1,"Phải")]  
    for dx, dy, Huong in BuocDi:  
        nx, ny = x + dx, y + dy  
        if 0 <= nx < 3 and 0 <= ny < 3:  
            Moi = [hang[:] for hang in TrangThai]  
            Moi[x][y], Moi[nx][ny] = Moi[nx][ny], Moi[x][y]  
            KetQua.append((Moi, Huong))  
    return KetQua
```

Di chuyển ô trống (0) để tạo ra các trạng thái hợp lệ.



IV. Thuật toán A*

```
def ASao(TrangThaiBanDau):
    HangCho = []
    heapq.heappush(HangCho, (KhoangCachManhattan(TrangThaiBanDau), 0, TrangThaiBanDau, []))
    DaXet = set()

    while HangCho:
        f, g, TrangThai, DuongDi = heapq.heappop(HangCho)
        h = KhoangCachManhattan(TrangThai)

        print("g =", g, ", h =", h, ", f =", f)
        for hang in TrangThai:
            print(hang)
        print("-"*20)

        if TrangThai == TrangThaiDich:
            print("✅ Đã tìm thấy trạng thái đích!")
            return DuongDi

        if DoiSangTuple(TrangThai) in DaXet:
            continue
        DaXet.add(DoiSangTuple(TrangThai))

        for Ke, Huong in HangXom(TrangThai):
            gMoi = g + 1
            hMoi = KhoangCachManhattan(Ke)
            fMoi = gMoi + hMoi
            heapq.heappush(HangCho, (fMoi, gMoi, Ke, DuongDi + [(Ke, Huong)]))
```

- Khởi tạo HangCho, DaXet.
- Vòng lặp: lấy trạng thái tốt nhất, in thông tin, kiểm tra đích, sinh trạng thái mới.



V.Chạy thử

```
TrangThaiBanDau = [[1, 2, 3],
                    [4, 5, 0],
                    [7, 8, 6]]

LoiGiai = ASao(TrangThaiBanDau)

if LoiGiai:
    for i, (s, Buoc) in enumerate(LoiGiai):
        print(f"Bước {i+1}: Di chuyển {Buoc}")
        for hang in s:
            print(hang)
        print()
    else:
        print("Không tìm được lời giải")
```



```
g = 0 , h = 1 , f = 1
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]
-----
g = 1 , h = 0 , f = 1
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
-----
✅ Đã tìm thấy trạng thái đích!
Bước 1: Di chuyển Xuống
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

- Gán trạng thái ban đầu.
- Gọi hàm ASao.
- In ra các bước di chuyển nếu tìm được lời giải.

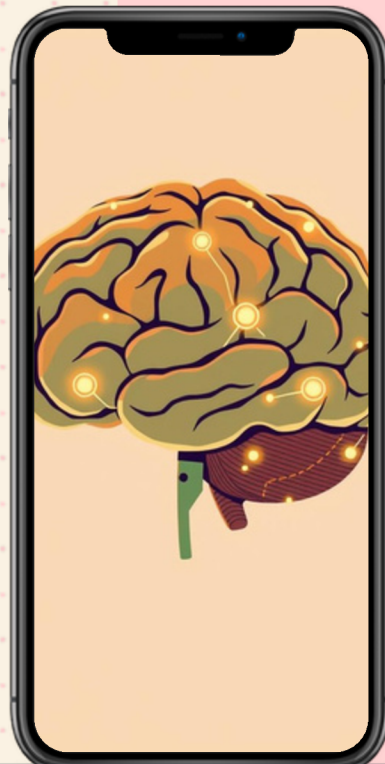
Kết quả

KẾT LUẬN

Bài toán 8-puzzle là một thách thức kinh điển trong lĩnh vực trí tuệ nhân tạo, đòi hỏi sự áp dụng hiệu quả của các thuật toán tìm kiếm. Trong báo cáo này, chúng em đã trình bày chi tiết về cách sử dụng thuật toán A* để giải quyết bài toán 8-puzzle, từ việc mô tả thuật toán, cài đặt đến việc đánh giá các hàm heuristic khác nhau. Thuật toán A* đã chứng minh được hiệu quả vượt trội trong việc tìm kiếm đường đi ngắn nhất từ trạng thái ban đầu đến trạng thái đích trong bài toán 8-puzzle. Điều này đạt được nhờ vào việc kết hợp chi phí thực tế từ điểm khởi đầu đến điểm hiện tại ($g(n)$) và chi phí ước lượng từ điểm hiện tại đến điểm đích ($h(n)$).

Qua quá trình nghiên cứu và thực hiện đề tài, chúng em đã rút ra được những bài học quý báu về cách áp dụng thuật toán A* trong việc giải quyết các bài toán tìm kiếm.

Kết quả của đề tài không chỉ dừng lại ở việc giải quyết bài toán 8-puzzle mà còn mở ra những hướng nghiên cứu và ứng dụng rộng hơn trong các lĩnh vực khác như lập kế hoạch di chuyển cho robot, phát triển các trò chơi giải đố, và tối ưu hóa đường đi trong hệ thống tự động hóa.





CẢM ƠN THẦY VÀ
CÁC BẠN ĐÃ LẮNG
NGHE BÀI THUYẾT
TRÌNH CỦA NHÓM

